

Modelos Auto Regresivos

Modelos Auto Regresivos

Jan Polanco Velasco

- Regresión Lineal

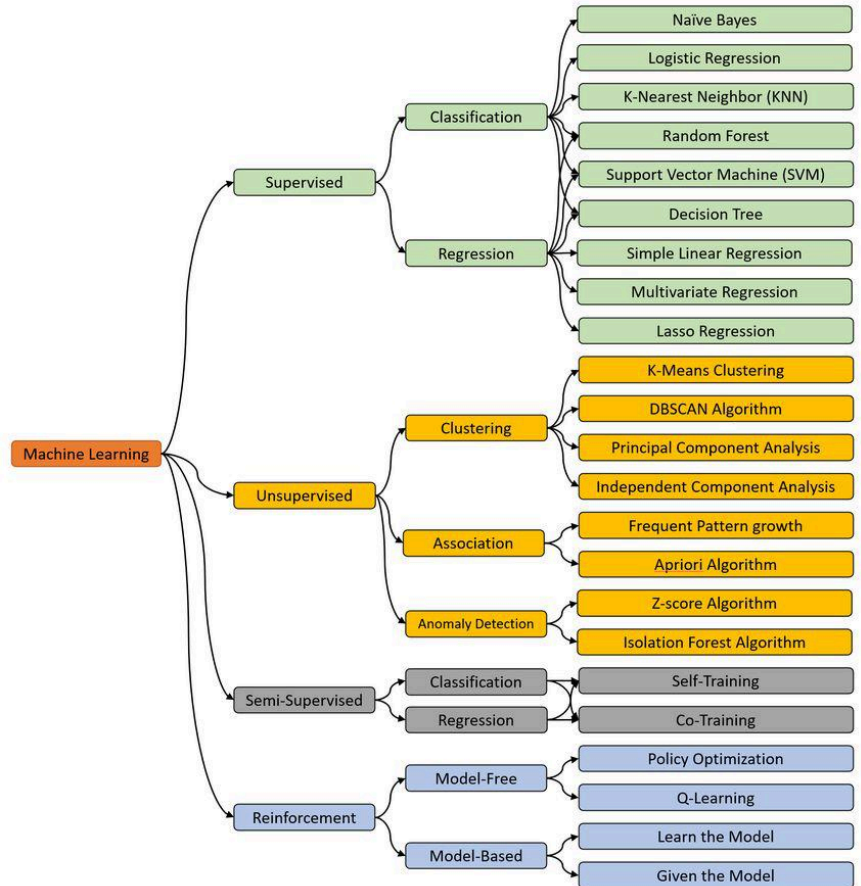
- Regresión Lineal
- Función de costo

- Regresión Lineal
- Función de costo
- Gradiente descendente

- Regresión Lineal
- Función de costo
- Gradiente descendente
- Regresión Lineal multivariada

- Regresión Lineal
- Función de costo
- Gradiente descendente
- Regresión Lineal multivariada
- Regresión Logística

- Regresión Lineal
- Función de costo
- Gradiente descendente
- Regresión Lineal multivariada
- Regresión Logística



layout: two-cols

Regresión Lineal

Regresión Lineal

- **Relación lineal:** Este modelo describe una relación lineal entre las entradas x y las salidas y .

Regresión Lineal

- **Relación lineal:** Este modelo describe una relación lineal entre las entradas x y las salidas y .
- Donde w_0 y w_1 (*weights*) representan el intercepto y la pendiente de la recta, respectivamente.

Regresión Lineal

- **Relación lineal:** Este modelo describe una relación lineal entre las entradas x y las salidas y .
- Donde w_0 y w_1 (*weights*) representan el intercepto y la pendiente de la recta, respectivamente.
- Los pesos pueden tomar cualquier valor y diferentes pesos producen diferentes rectas.

Regresión Lineal

- **Relación lineal:** Este modelo describe una relación lineal entre las entradas x y las salidas y .
- Donde w_0 y w_1 (*weights*) representan el intercepto y la pendiente de la recta, respectivamente.
- Los pesos pueden tomar cualquier valor y diferentes pesos producen diferentes rectas.
- Es necesario calcular la función de costo (*para encontrar los mejores pesos*).

Regresión Lineal

- **Relación lineal:** Este modelo describe una relación lineal entre las entradas x y las salidas y .
- Donde w_0 y w_1 (*weights*) representan el intercepto y la pendiente de la recta, respectivamente.
- Los pesos pueden tomar cualquier valor y diferentes pesos producen diferentes rectas.
- Es necesario calcular la función de costo (*para encontrar los mejores pesos*).

$$y = w_0 + w_1 \cdot x$$

Función de costos

Función de costos

- Los pesos pueden tomar cualquier valor.

Función de costos

- Los pesos pueden tomar cualquier valor.
- Diferentes pesos producen diferentes rectas.

Función de costos

- Los pesos pueden tomar cualquier valor.
- Diferentes pesos producen diferentes rectas.
- Es necesario calcular la función de costo (*Los mejores pesos*).

Función de costos

- Los pesos pueden tomar cualquier valor.
- Diferentes pesos producen diferentes rectas.
- Es necesario calcular la función de costo (*Los mejores pesos*).
- **La función de costo (asumiendo w_0 conocido):**

Función de costos

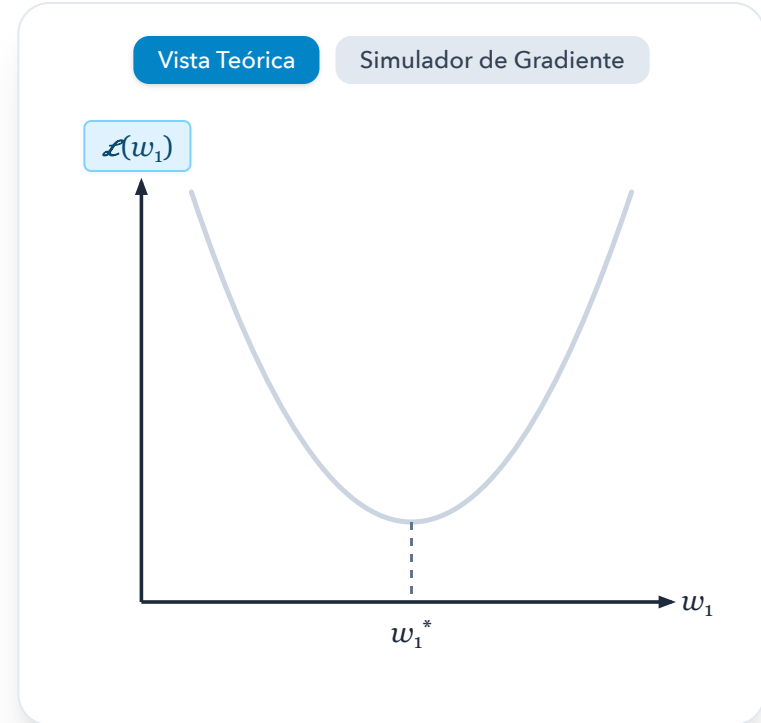
- Los pesos pueden tomar cualquier valor.
- Diferentes pesos producen diferentes rectas.
- Es necesario calcular la función de costo (*Los mejores pesos*).
- **La función de costo (asumiendo w_0 conocido):**

$$\mathcal{L}(w_1) = \sum_{i=1}^N (w_1 \cdot x_i - y_i)^2$$

Función de costos

- Los pesos pueden tomar cualquier valor.
- Diferentes pesos producen diferentes rectas.
- Es necesario calcular la función de costo (*Los mejores pesos*).
- **La función de costo (asumiendo w_0 conocido):**

$$\mathcal{L}(w_1) = \sum_{i=1}^N (w_1 \cdot x_i - y_i)^2$$



10 Most Common Loss Functions in Machine Learning

10 Most Common Loss Functions in Machine Learning

Regression Loss Functions

10 Most Common Loss Functions in Machine Learning

Regression Loss Functions

Loss Function	Description	Formula
Mean Bias Error (MBE)	Captures average bias in prediction. Rarely used for training.	$\mathcal{L}_{MBE} = \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))$
Mean Absolute Error (MAE / L1)	Measures absolute average bias in prediction.	$\mathcal{L}_{MAE} = \frac{1}{N} \sum_{i=1}^N y_i - f(x_i) $
Mean Squared Error (MSE / L2)	Average squared distance between actual and predicted.	$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))^2$
Root Mean Squared Error (RMSE)	Square root of MSE. Same units as target.	$\mathcal{L}_{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))^2}$

10 Most Common Loss Functions in Machine Learning

10 Most Common Loss Functions in Machine Learning

Regression Loss Functions

10 Most Common Loss Functions in Machine Learning

Regression Loss Functions

Loss Function	Description	Formula
Huber Loss	Combination of MSE and MAE. Parametric robust loss.	$\mathcal{L}_\delta = \begin{cases} \frac{1}{2}(y - f(x))^2 & \text{si } y - f(x) \leq \delta \\ \delta(y - f(x) - \frac{1}{2}\delta) & \text{en otro caso} \end{cases}$
Log Cosh Loss	Similar to Huber, non-parametric, twice differentiable.	$\mathcal{L}_{\log \cosh} = \sum_{i=1}^N \log(\cosh(f(x_i) - y_i))$

10 Most Common Loss Functions in Machine Learning

10 Most Common Loss Functions in Machine Learning

Classification Loss Functions

10 Most Common Loss Functions in Machine Learning

Classification Loss Functions

Loss Function	Description	Formula
Binary Cross Entropy (BCE)	Loss function for binary classification tasks.	$\mathcal{L}_{BCE} = -\frac{1}{N} \sum [y_i \log p(x_i) + (1 - y_i) \log(1 - p(x_i))]$
Hinge Loss	Penalizes wrong and low-confidence predictions (SVMs).	$\mathcal{L}_{\text{hinge}} = \max(0, 1 - f(x) \cdot y)$
Categorical Cross Entropy	Multi-class classification extension of BCE.	$\mathcal{L}_{CE} = -\frac{1}{N} \sum \sum y_{ij} \log(f(x_{ij}))$
KL Divergence	Relative entropy between true and predicted distributions.	$\mathcal{L}_{KL} = \sum y_i \cdot \log \left(\frac{y_i}{f(x_i)} \right)$

Gradiente Descendente

Gradiente Descendente

$$\mathcal{L}(w_1) = \sum_{i=1}^N (w_1 \cdot x_i - y_i)^2$$

Gradiente Descendente

$$\mathcal{L}(w_1) = \sum_{i=1}^N (w_1 \cdot x_i - y_i)^2$$

$$w^{(t+1)} = w^{(t)} - \alpha \nabla f_i(w^{(t)})$$

Gradiente Descendente

$$\mathcal{L}(w_1) = \sum_{i=1}^N (w_1 \cdot x_i - y_i)^2$$

$$w^{(t+1)} = w^{(t)} - \alpha \nabla f_i(w^{(t)})$$

- $w^{(t+1)}$: Posición de la siguiente iteración

Gradiente Descendente

$$\mathcal{L}(w_1) = \sum_{i=1}^N (w_1 \cdot x_i - y_i)^2$$

$$w^{(t+1)} = w^{(t)} - \alpha \nabla f_i(w^{(t)})$$

- $w^{(t+1)}$: Posición de la siguiente iteración
- $w^{(t)}$: Posición del paso anterior

Gradiente Descendente

$$\mathcal{L}(w_1) = \sum_{i=1}^N (w_1 \cdot x_i - y_i)^2$$

$$w^{(t+1)} = w^{(t)} - \alpha \nabla f_i(w^{(t)})$$

- $w^{(t+1)}$: Posición de la siguiente iteración
- $w^{(t)}$: Posición del paso anterior
- α : Tasa de aprendizaje (*learning rate / step size*)

Gradiente Descendente

$$\mathcal{L}(w_1) = \sum_{i=1}^N (w_1 \cdot x_i - y_i)^2$$

$$w^{(t+1)} = w^{(t)} - \alpha \nabla f_i(w^{(t)})$$

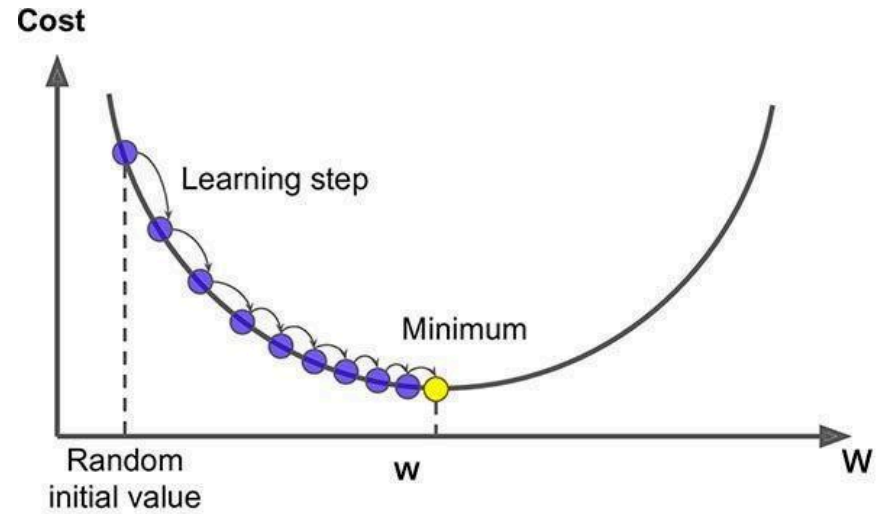
- $w^{(t+1)}$: Posición de la siguiente iteración
- $w^{(t)}$: Posición del paso anterior
- α : Tasa de aprendizaje (*learning rate / step size*)
- $\nabla f_i(w^{(t)})$: Gradiente en la observación i

Gradiente Descendente

$$\mathcal{L}(w_1) = \sum_{i=1}^N (w_1 \cdot x_i - y_i)^2$$

$$w^{(t+1)} = w^{(t)} - \alpha \nabla f_i(w^{(t)})$$

- $w^{(t+1)}$: Posición de la siguiente iteración
- $w^{(t)}$: Posición del paso anterior
- α : Tasa de aprendizaje (*learning rate / step size*)
- $\nabla f_i(w^{(t)})$: Gradiente en la observación i



Superficies de Costo No Convexas

Superficies de Costo No Convexas

- En redes profundas $J(\theta_0, \theta_1)$ ya no es un tazón: aparecen **mínimos locales**, **puntos de silla** (*saddle points*) y valles planos.

Superficies de Costo No Convexas

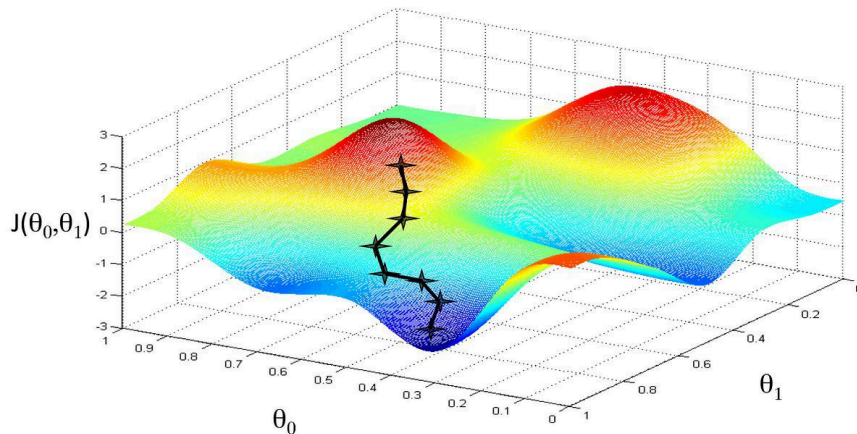
- En redes profundas $J(\theta_0, \theta_1)$ ya no es un tazón: aparecen **mínimos locales**, **puntos de silla** (*saddle points*) y valles planos.
- El gradiente descendente converge al mínimo de **su propia cuenca de atracción**, no necesariamente al mínimo global.

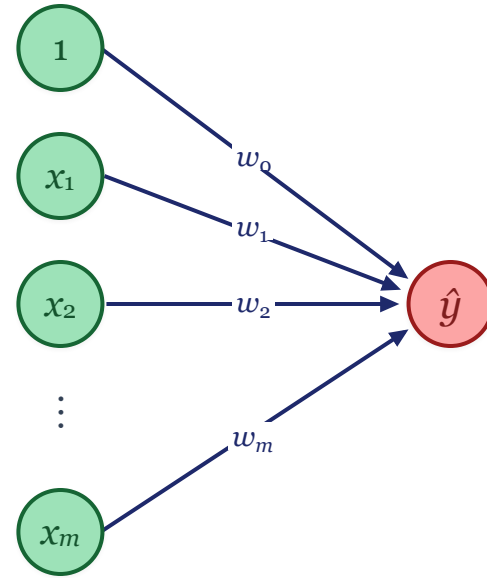
Superficies de Costo No Convexas

- En redes profundas $J(\theta_0, \theta_1)$ ya no es un tazón: aparecen **mínimos locales**, **puntos de silla** (*saddle points*) y valles planos.
- El gradiente descendente converge al mínimo de **su propia cuenca de atracción**, no necesariamente al mínimo global.
- La trayectoria depende de la inicialización $w^{(0)}$ y de α : pasos grandes oscilan, pasos pequeños se estancan.

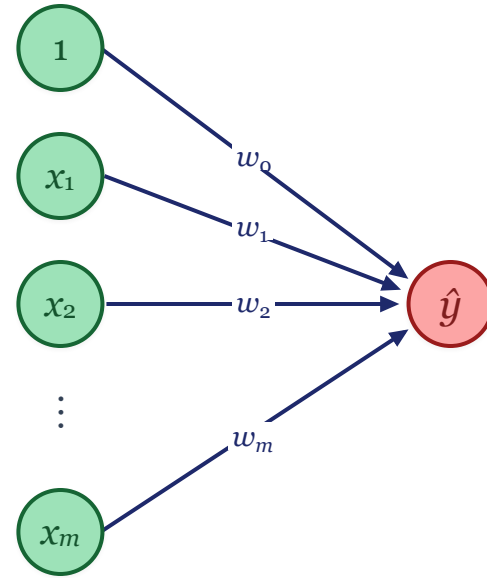
Superficies de Costo No Convexas

- En redes profundas $J(\theta_0, \theta_1)$ ya no es un tazón: aparecen **mínimos locales**, **puntos de silla** (*saddle points*) y valles planos.
- El gradiente descendente converge al mínimo de **su propia cuenca de atracción**, no necesariamente al mínimo global.
- La trayectoria depende de la inicialización $w^{(0)}$ y de α : pasos grandes oscilan, pasos pequeños se estancan.



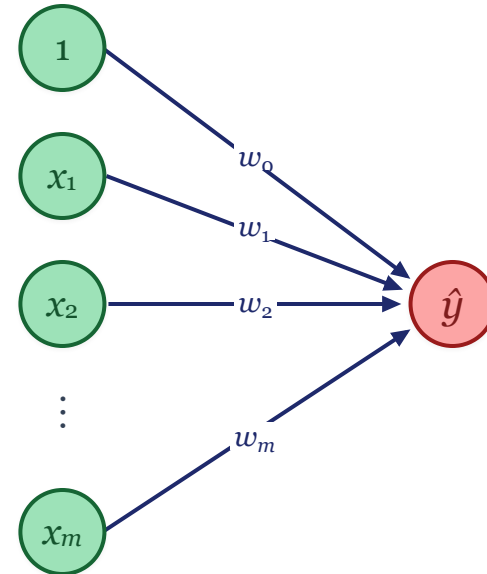


Regresión Lineal Multivariada



Regresión Lineal Multivariada

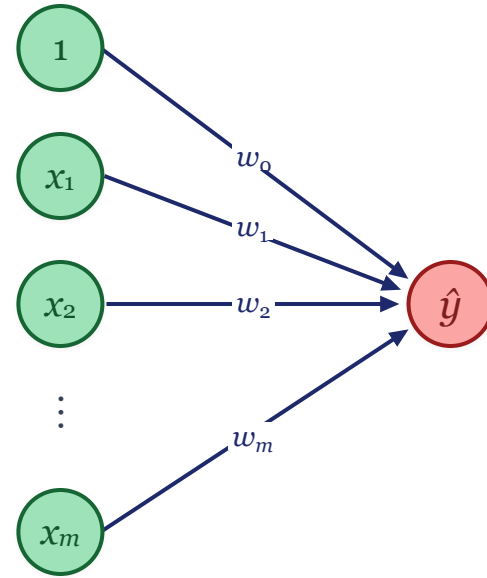
- Se puede aplicar a datos con **múltiples atributos**.
- Los parámetros del modelo se estiman a partir de los conceptos de **Función de costo** y **Gradiente descendente**.
- En este caso, la predicción se calcula como una **combinación lineal** de todos los atributos de entrada.



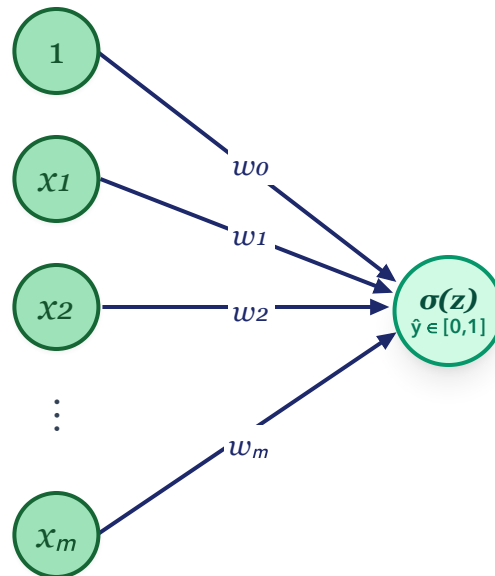
Regresión Lineal Multivariada

- Se puede aplicar a datos con **múltiples atributos**.
- Los parámetros del modelo se estiman a partir de los conceptos de **Función de costo** y **Gradiente descendente**.
- En este caso, la predicción se calcula como una **combinación lineal** de todos los atributos de entrada.

$$\hat{y} = w_0 + \sum_{j=1}^m w_j \cdot x_j$$

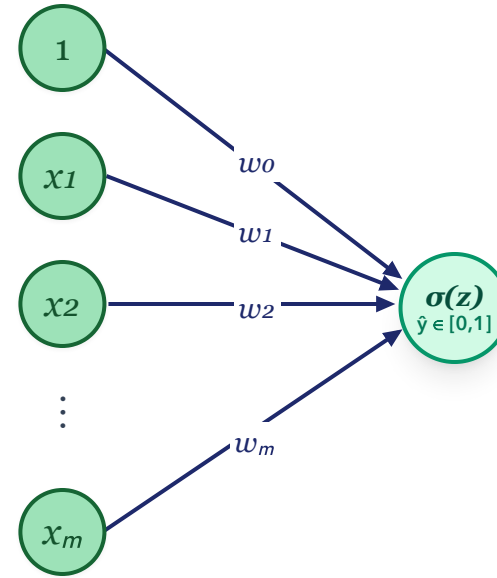


Regresión Logística



Regresión Logística

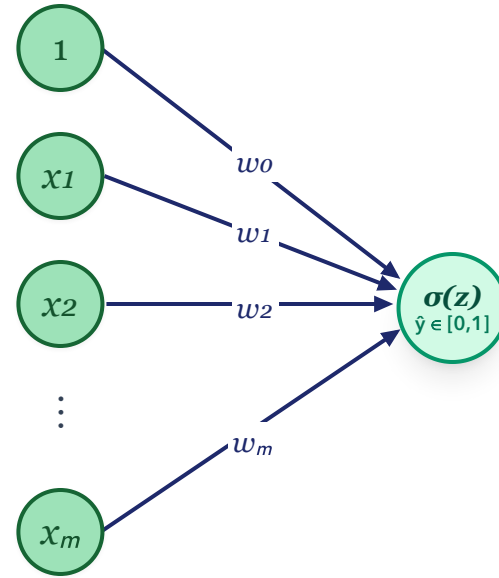
- **Clasificación binaria:** Modela la probabilidad de que una instancia pertenezca a la clase positiva ($y = 1$).
- **Combinación lineal acotada:** Pasa la pre-activación z por la función sigmoide $\sigma(z)$ para garantizar un rango en $[0, 1]$.



Regresión Logística

- **Clasificación binaria:** Modela la probabilidad de que una instancia pertenezca a la clase positiva ($y = 1$).
- **Combinación lineal acotada:** Pasa la pre-activación z por la función sigmoide $\sigma(z)$ para garantizar un rango en $[0, 1]$.

$$\hat{y} = p(y = 1 \mid \mathbf{x}) = \sigma \left(w_0 + \sum_{j=1}^m w_j x_j \right)$$

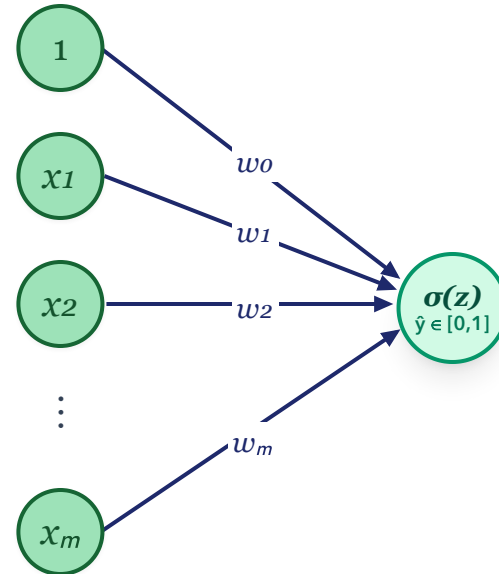


Regresión Logística

- **Clasificación binaria:** Modela la probabilidad de que una instancia pertenezca a la clase positiva ($y = 1$).
- **Combinación lineal acotada:** Pasa la pre-activación z por la función sigmoide $\sigma(z)$ para garantizar un rango en $[0, 1]$.

$$\hat{y} = p(y = 1 \mid \mathbf{x}) = \sigma \left(w_0 + \sum_{j=1}^m w_j x_j \right)$$

- **Función de costo:** En lugar de MSE, se optimiza mediante **Entropía Cruzada Binaria (BCE)**.



Red Neuronal Fully Connected

Red Neuronal Fully Connected

- **Capa de Entrada** (*Input Layer*)

Red Neuronal Fully Connected

- **Capa de Entrada** (*Input Layer*)
- **Capa oculta** (*Hidden Layer - h*)

Red Neuronal Fully Connected

- **Capa de Entrada** (*Input Layer*)
- **Capa oculta** (*Hidden Layer - h*)
- **Capa de salida** (*Output Layer*)

Red Neuronal Fully Connected

- **Capa de Entrada** (*Input Layer*)
- **Capa oculta** (*Hidden Layer - h*)
- **Capa de salida** (*Output Layer*)

¿Cuántas capas tiene la NN?

Red Neuronal Fully Connected

- **Capa de Entrada** (*Input Layer*)
- **Capa oculta** (*Hidden Layer - h*)
- **Capa de salida** (*Output Layer*)

¿Cuántas capas tiene la NN?



Neurona biológica

Dendritas reciben → Soma/núcleo integra → Axón transmite



Neurona artificial (McCulloch-Pitts)

Suma ponderada de entradas $\times$ pesos + sesgo, luego activación $g(\cdot)$

Red Neuronal Fully Connected

Red Neuronal Fully Connected

- La capa de entrada **no suele considerarse** en el conteo de capas.

Red Neuronal Fully Connected

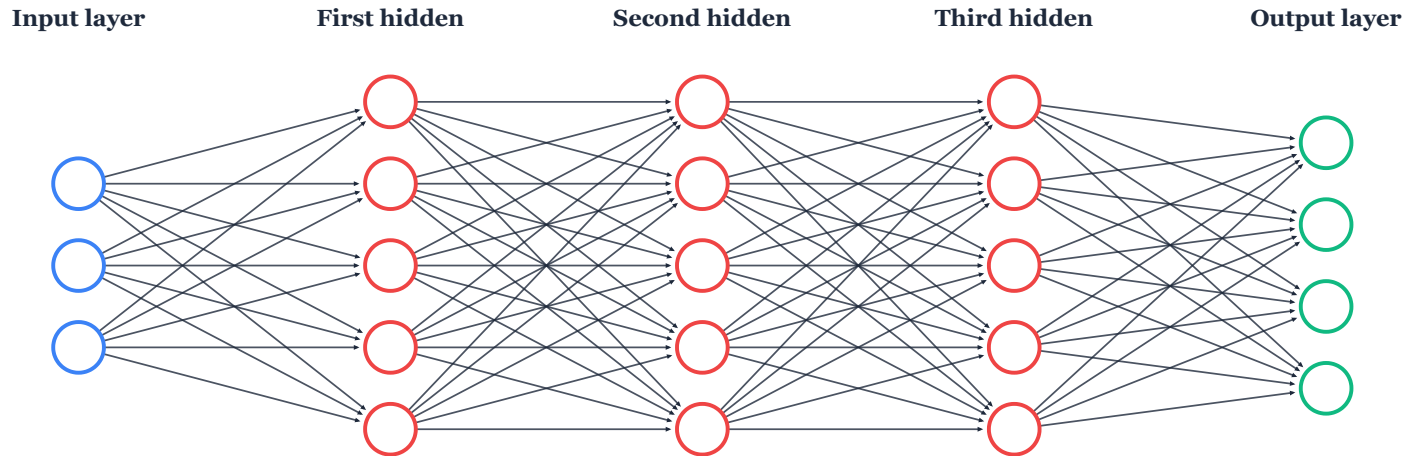
- La capa de entrada **no suele considerarse** en el conteo de capas.
- **Tiene 4 capas:** $L = 4$ (*3 capas ocultas + 1 capa de salida*).

Red Neuronal Fully Connected

- La capa de entrada **no suele considerarse** en el conteo de capas.
- Tiene 4 capas:** $L = 4$ (3 capas ocultas + 1 capa de salida).

Total parámetros: **85 conexiones** (15 + 25 + 25 + 20)

⚡ Simular Forward Pass



Capas Ocultas y Notación Matricial

Capas Ocultas y Notación Matricial

- El **superíndice** indica la capa.

Capas Ocultas y Notación Matricial

- El **superíndice** indica la capa.
- h_1 necesita $3 + 1$ parámetros.

Capas Ocultas y Notación Matricial

- El **superíndice** indica la capa.
- h_1 necesita $3 + 1$ parámetros.
- La primera capa oculta necesita 4×5 parámetros.

Capas Ocultas y Notación Matricial

- El **superíndice** indica la capa.
- h_1 necesita $3 + 1$ parámetros.
- La primera capa oculta necesita 4×5 parámetros.

Ecuación general para h_1 :

$$h_1 = g \left(w_{1,0}^{[1]} + \sum_{j=1}^3 w_{1,j}^{[1]} x_j \right)$$

- $g(\cdot)$: Función de activación (ReLU, sigmoide, etc.)
- $w_{1,0}^{[1]}$: Bias o término de sesgo.
- $w_{1,j}^{[1]}$: Peso asociado a cada entrada x_j .

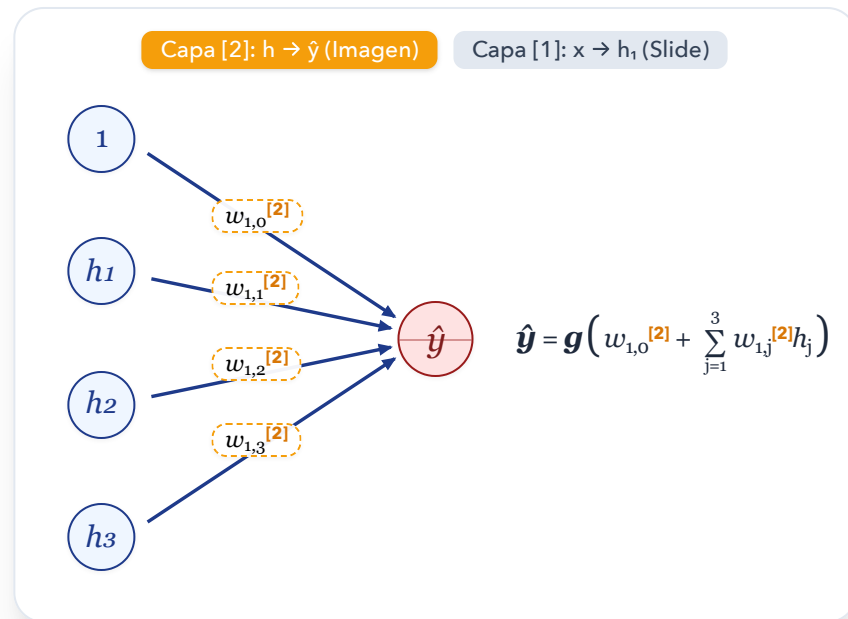
Capas Ocultas y Notación Matricial

- El **superíndice** indica la capa.
- h_1 necesita $3 + 1$ parámetros.
- La primera capa oculta necesita 4×5 parámetros.

Ecuación general para h_1 :

$$h_1 = g \left(w_{1,0}^{[1]} + \sum_{j=1}^3 w_{1,j}^{[1]} x_j \right)$$

- $g(\cdot)$: Función de activación (ReLU, sigmoide, etc.)
- $w_{1,0}^{[1]}$: Bias o término de sesgo.
- $w_{1,j}^{[1]}$: Peso asociado a cada entrada x_j .



Cálculo de la neurona de salida $\hat{y}$

Cálculo de la neurona de salida $\hat{y}$

$$\hat{y} = g \left(w_{1,0}^{[2]} + \sum_{j=1}^3 w_{1,j}^{[2]} h_j \right)$$

Cálculo de la neurona de salida $\hat{y}$

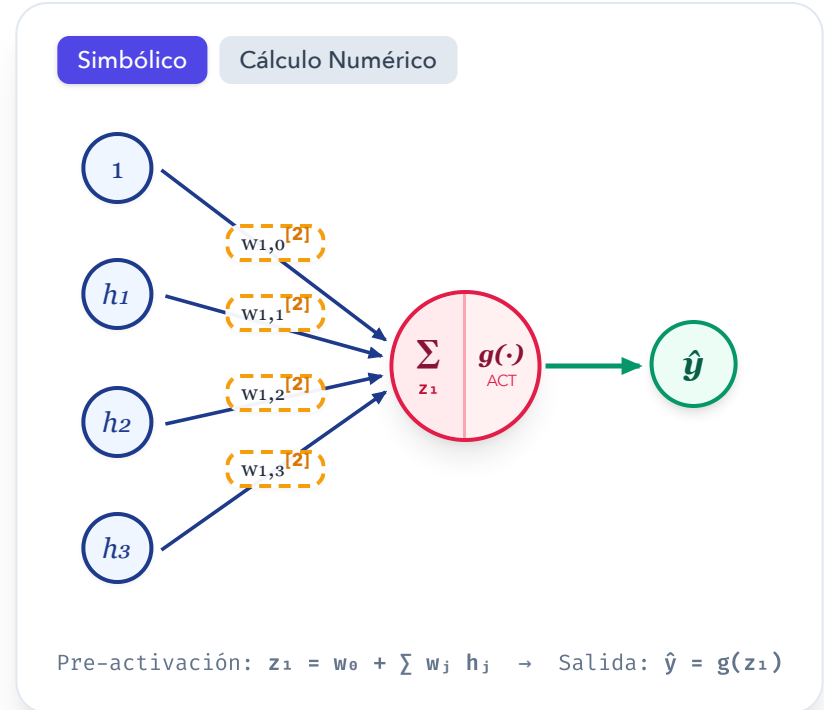
$$\hat{y} = g \left(w_{1,0}^{[2]} + \sum_{j=1}^3 w_{1,j}^{[2]} h_j \right)$$

- Ponderación de los estados ocultos h_1, h_2, h_3 junto al sesgo $w_{1,0}^{[2]}$.

Cálculo de la neurona de salida $\hat{y}$

$$\hat{y} = g \left(w_{1,0}^{[2]} + \sum_{j=1}^3 w_{1,j}^{[2]} h_j \right)$$

- Ponderación de los estados ocultos h_1, h_2, h_3 junto al sesgo $w_{1,0}^{[2]}$.



Funciones de Activación

Funciones de Activación

- **Añaden No Linealidad:** Las funciones de activación son esenciales para agregar un componente no lineal en las redes neuronales, permitiendo que estas redes puedan aprender y modelar relaciones complejas en los datos.

Funciones de Activación

- **Añaden No Linealidad:** Las funciones de activación son esenciales para agregar un componente no lineal en las redes neuronales, permitiendo que estas redes puedan aprender y modelar relaciones complejas en los datos.
- **Modelo Lineal sin Activación:** Sin funciones de activación, una red neuronal se reduce a un simple modelo lineal, sin importar cuántas capas ocultas contenga.

Funciones de Activación

- **Añaden No Linealidad:** Las funciones de activación son esenciales para agregar un componente no lineal en las redes neuronales, permitiendo que estas redes puedan aprender y modelar relaciones complejas en los datos.
- **Modelo Lineal sin Activación:** Sin funciones de activación, una red neuronal se reduce a un simple modelo lineal, sin importar cuántas capas ocultas contenga.
- **En Cualquier Capa:** Las funciones de activación pueden colocarse en cualquier capa de la red.

Funciones de Activación

Funciones de Activación

- Ciertas funciones son más apropiadas para **capas de salida**.

Funciones de Activación

- Ciertas funciones son más apropiadas para **capas de salida**.
- Otras son mejores para **capas ocultas**, optimizando el rendimiento de la red.

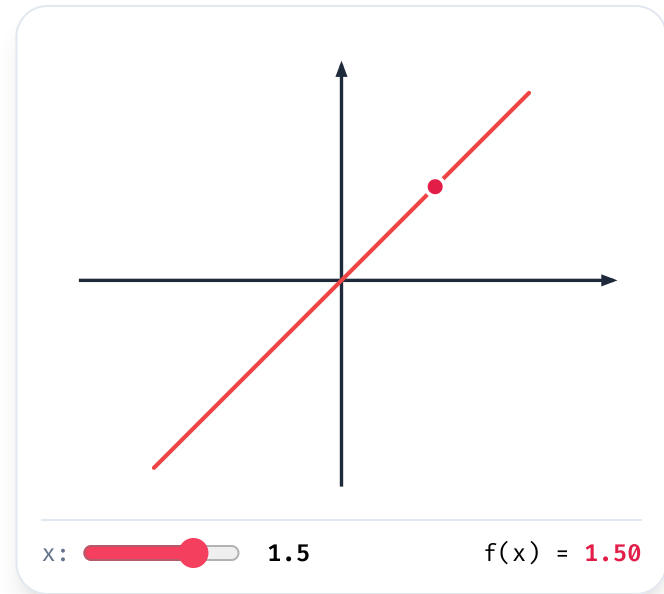
Funciones de Activación

- Ciertas funciones son más apropiadas para **capas de salida**.
- Otras son mejores para **capas ocultas**, optimizando el rendimiento de la red.

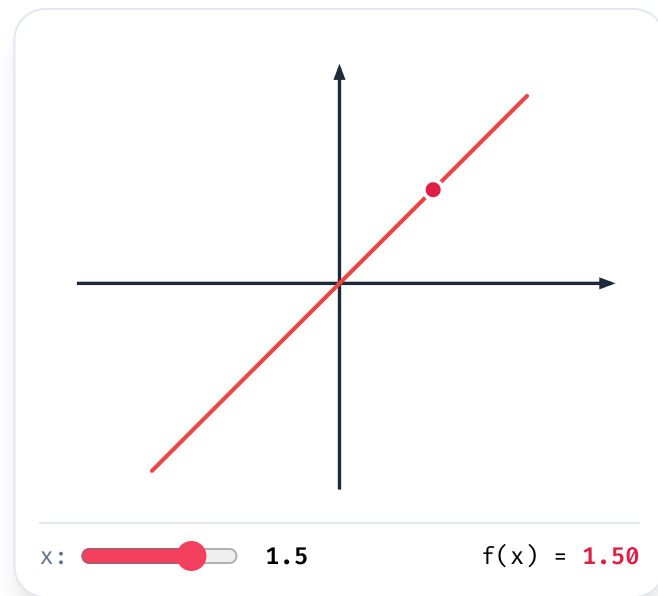
Definición Matemática

Para una unidad en una capa oculta o de salida, el cálculo del valor se realiza mediante la función de activación $g(\cdot)$:

$$h_2 = g \left(w_{2,0}^{[1]} + \sum_{j=1}^4 w_{2,j}^{[1]} x_j \right)$$

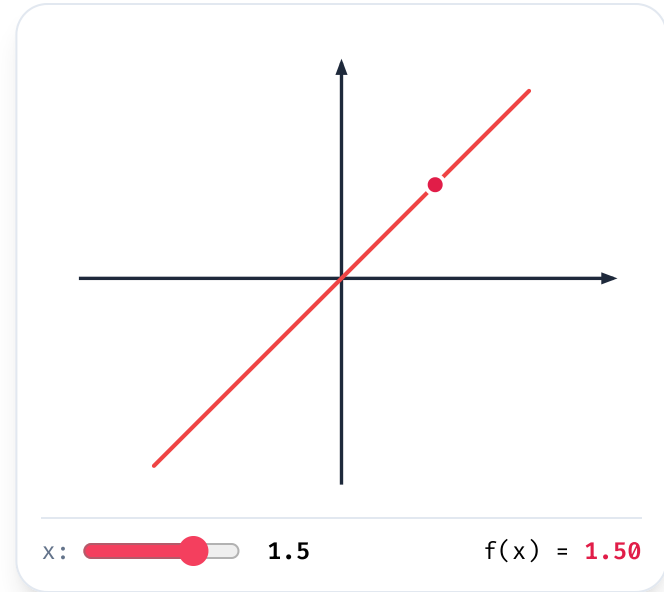


Tipos de Funciones de Activación: Identidad



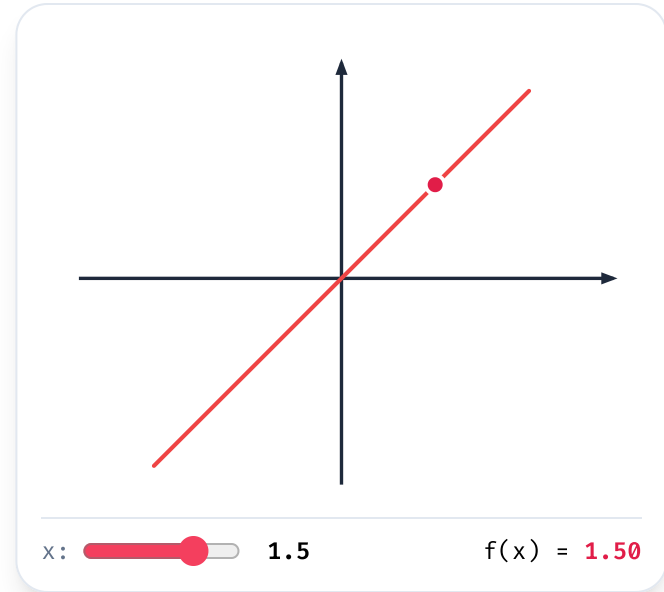
Tipos de Funciones de Activación: Identidad

- Función Identidad: $f(x) = x$



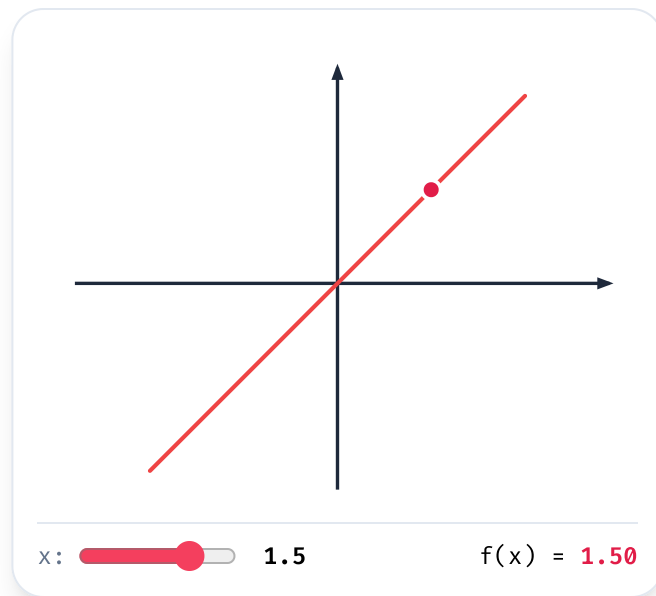
Tipos de Funciones de Activación: Identidad

- **Función Identidad:** $f(x) = x$
- Salida igual a la entrada.



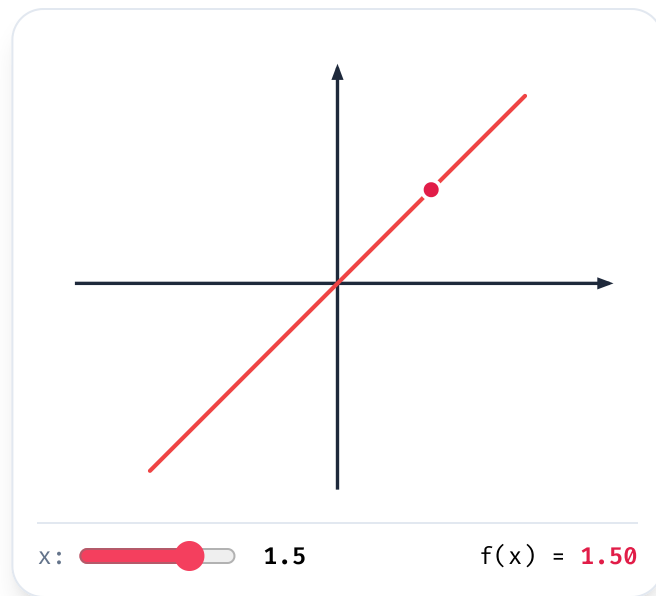
Tipos de Funciones de Activación: Identidad

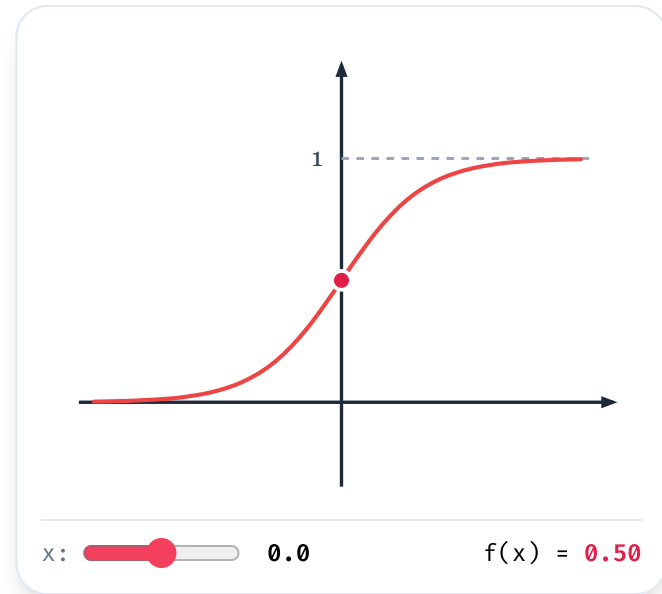
- **Función Identidad:** $f(x) = x$
- Salida igual a la entrada.
- En capas ocultas produce un modelo puramente lineal (*pierde la capacidad de aproximar no linealidades*).



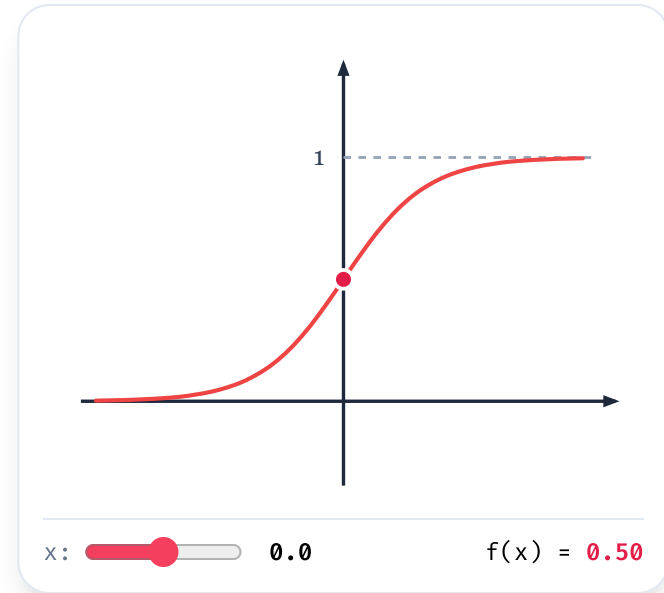
Tipos de Funciones de Activación: Identidad

- **Función Identidad:** $f(x) = x$
- Salida igual a la entrada.
- En capas ocultas produce un modelo puramente lineal (*pierde la capacidad de aproximar no linealidades*).
- **Se suele usar en la capa de salida** para problemas de regresión continua.



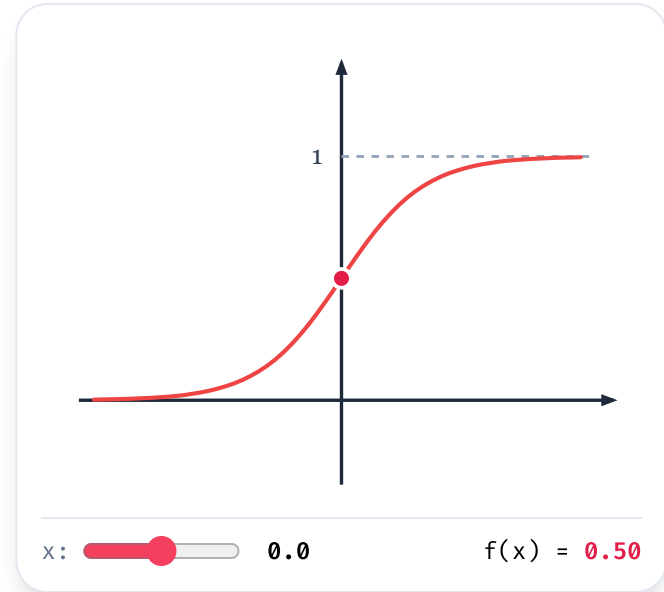


Tipos de Funciones de Activación: Sigmoides



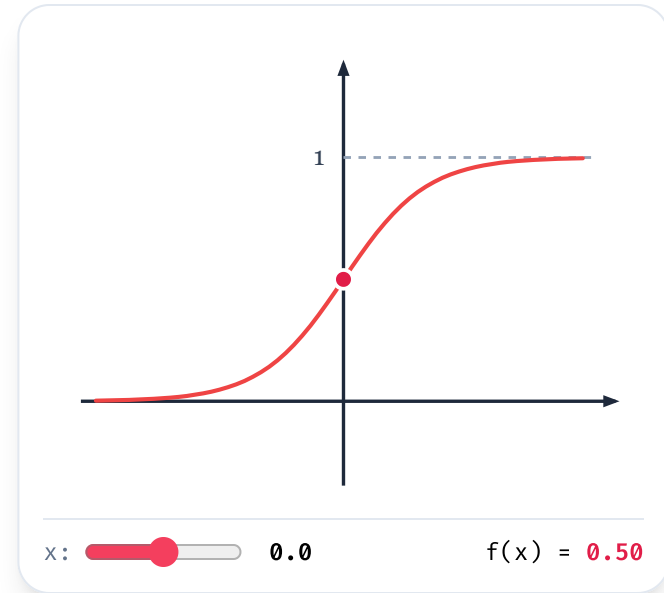
Tipos de Funciones de Activación: Sigmoides

- Función sigmoide: $\sigma(x) = \frac{1}{1+e^{-x}}$



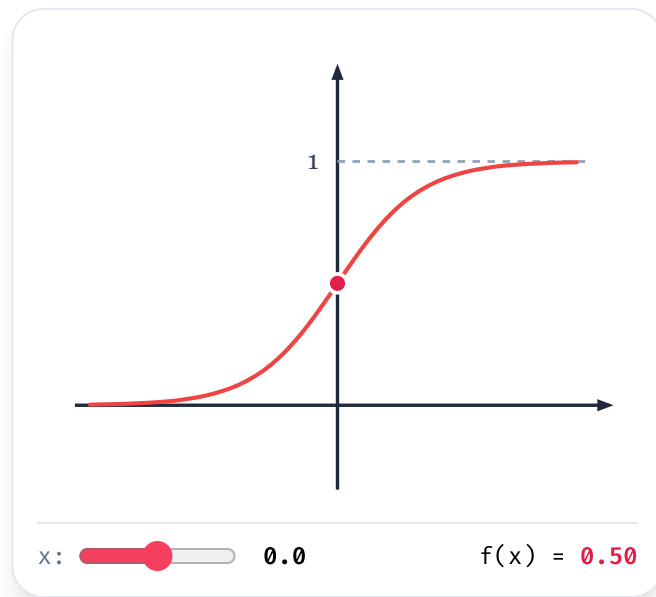
Tipos de Funciones de Activación: Sigmoides

- **Función sigmoide:** $\sigma(x) = \frac{1}{1+e^{-x}}$
- **Rango de salida:** $[0, 1]$ (*ideal para interpretar probabilidades*).



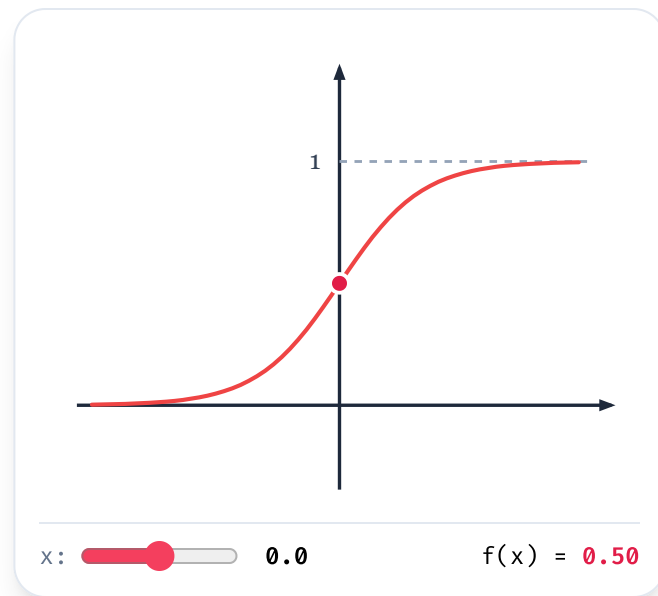
Tipos de Funciones de Activación: Sigmoides

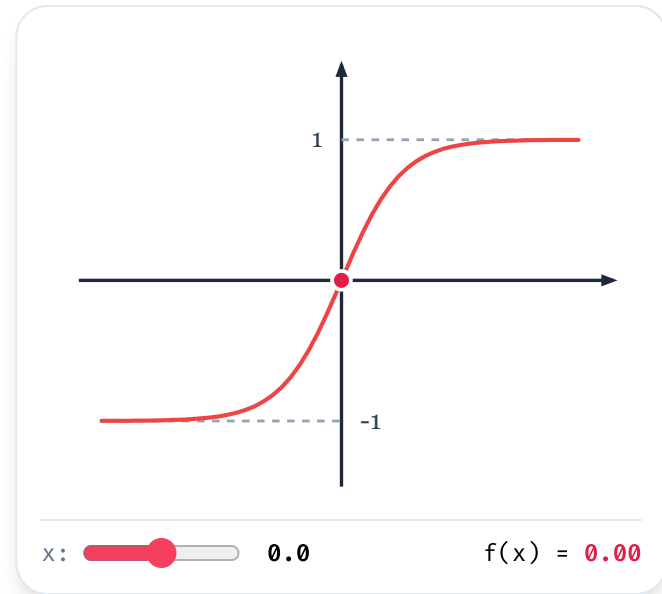
- **Función sigmoide:** $\sigma(x) = \frac{1}{1+e^{-x}}$
- **Rango de salida:** $[0, 1]$ (*ideal para interpretar probabilidades*).
- Introduce problemas de **saturación de gradientes** en valores extremos ($|x| > 3$).



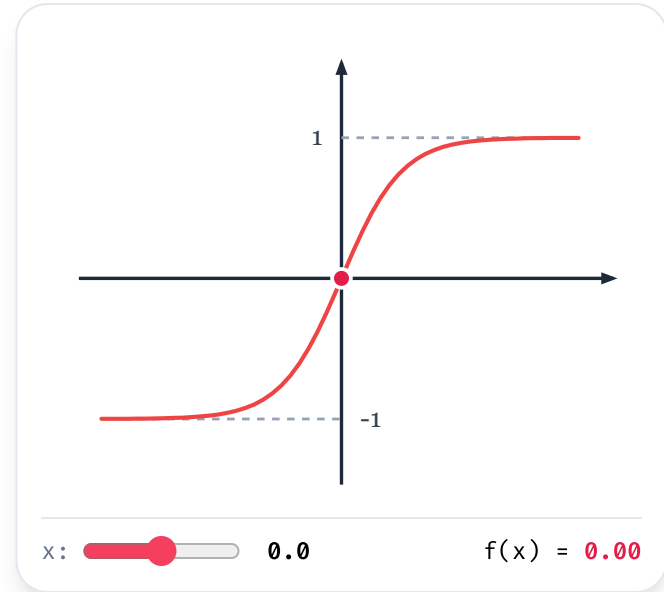
Tipos de Funciones de Activación: Sigmoide

- **Función sigmoide:** $\sigma(x) = \frac{1}{1+e^{-x}}$
- **Rango de salida:** $[0, 1]$ (ideal para interpretar probabilidades).
- Introduce problemas de **saturación de gradientes** en valores extremos ($|x| > 3$).
- **Softmax** es la versión generalizada para clasificación multiclase en la capa de salida.



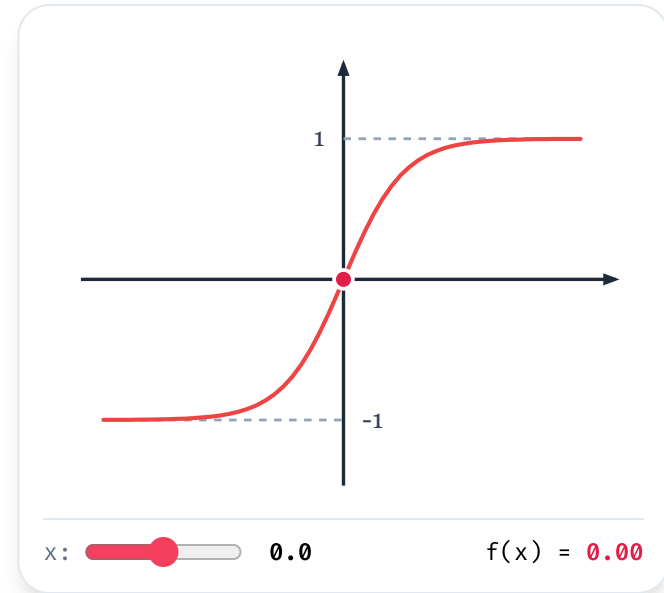


Tipos de Funciones de Activación: Tangente Hiperbólica



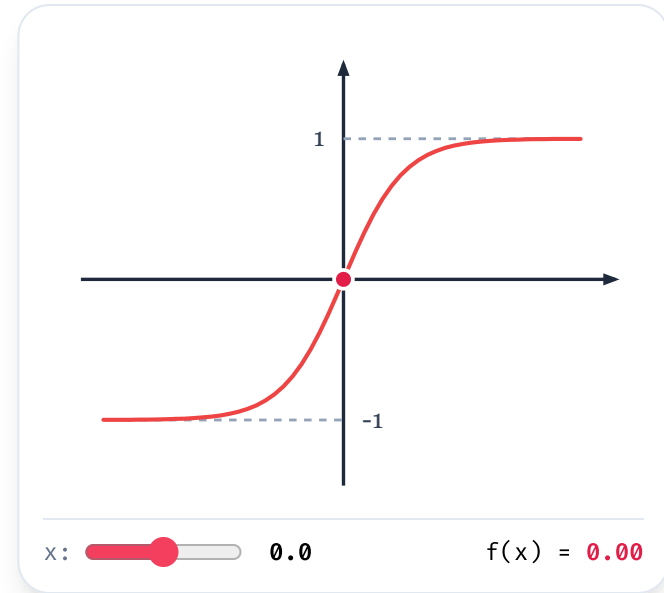
Tipos de Funciones de Activación: Tangente Hiperbólica

- Función Tangente Hiperbólica: $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$



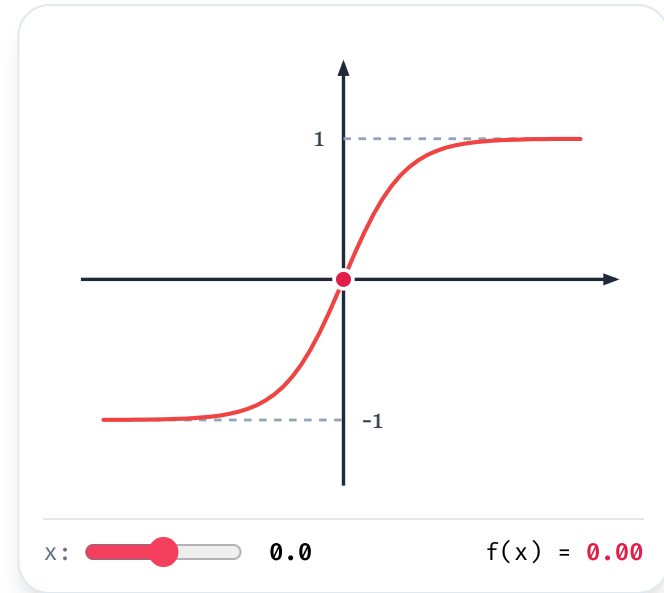
Tipos de Funciones de Activación: Tangente Hiperbólica

- Función Tangente Hiperbólica: $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- Rango de salida: $[-1, 1]$.



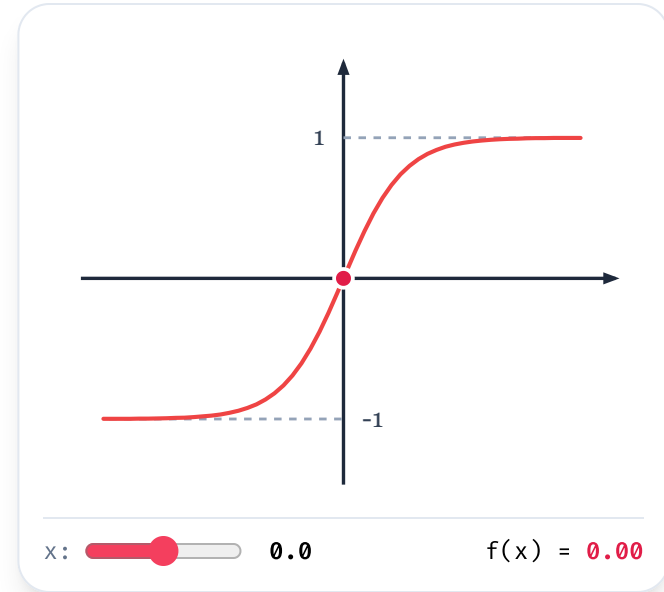
Tipos de Funciones de Activación: Tangente Hiperbólica

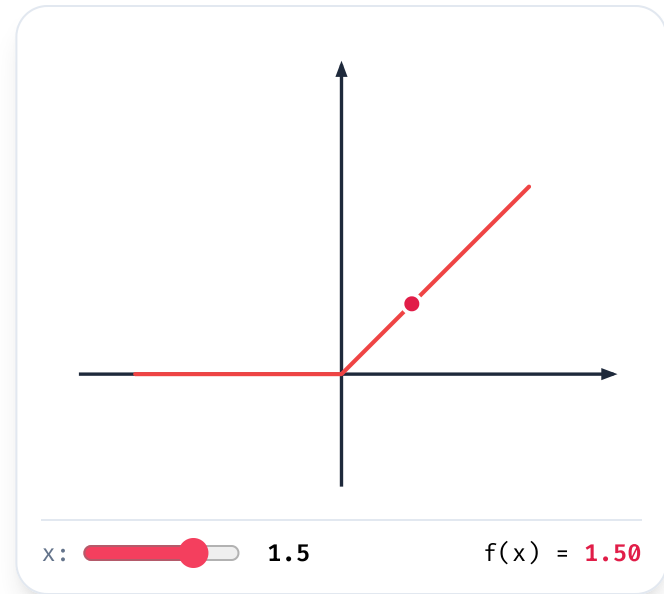
- **Función Tangente Hiperbólica:** $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- **Rango de salida:** $[-1, 1]$.
- **Centrada en cero:** Facilita la convergencia del entrenamiento frente a la sigmoide.



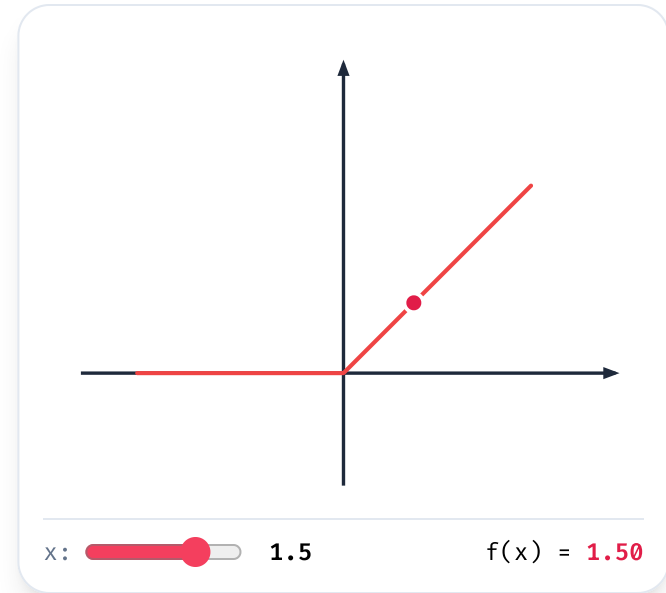
Tipos de Funciones de Activación: Tangente Hiperbólica

- **Función Tangente Hiperbólica:** $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- **Rango de salida:** $[-1, 1]$.
- **Centrada en cero:** Facilita la convergencia del entrenamiento frente a la sigmoide.
- Sufre del problema de **gradientes que se desvanecen** (*vanishing gradient*) en las colas.





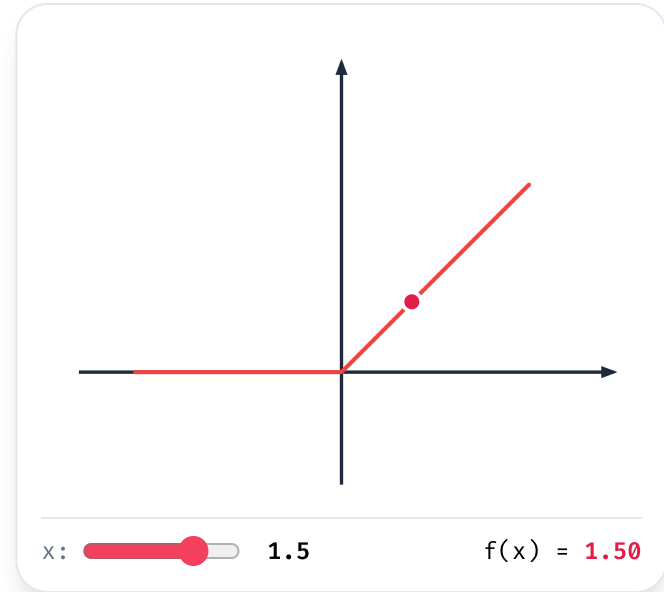
Tipos de Funciones de Activación: ReLU



Tipos de Funciones de Activación: ReLU

- **Función ReLU** (*Rectified Linear Unit*):

$$f(x) = \max(0, x)$$

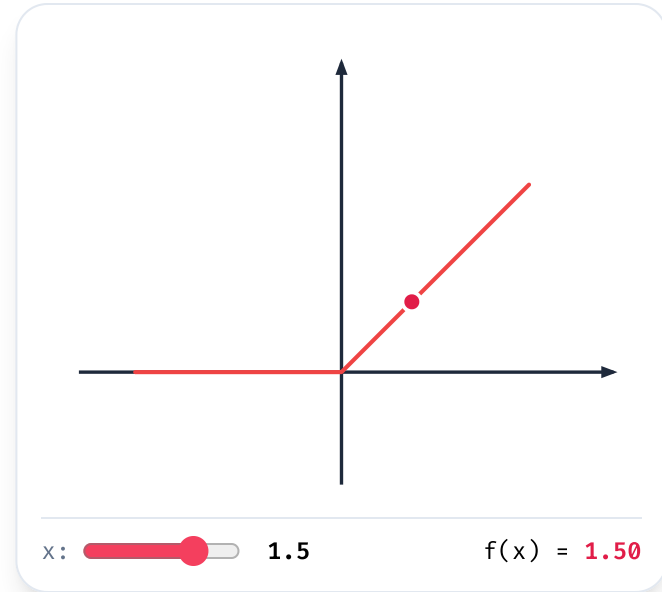


Tipos de Funciones de Activación: ReLU

- **Función ReLU** (*Rectified Linear Unit*):

$$f(x) = \max(0, x)$$

- Computacionalmente muy eficiente: cálculo directo sin exponenciales.

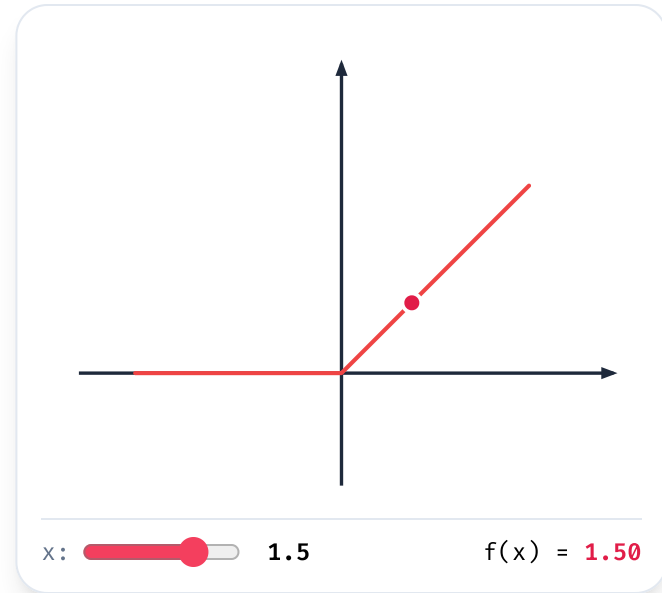


Tipos de Funciones de Activación: ReLU

- **Función ReLU** (*Rectified Linear Unit*):

$$f(x) = \max(0, x)$$

- Computacionalmente muy eficiente: cálculo directo sin exponenciales.
- Evita la saturación del gradiente para $x > 0$.

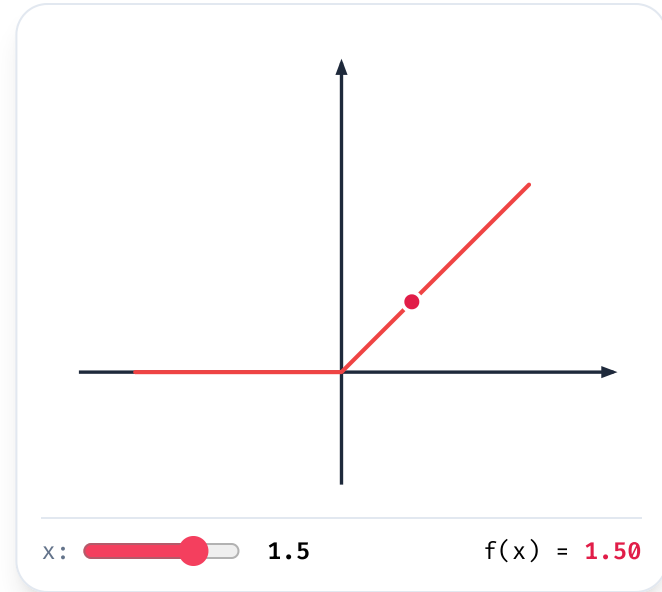


Tipos de Funciones de Activación: ReLU

- **Función ReLU** (*Rectified Linear Unit*):

$$f(x) = \max(0, x)$$

- Computacionalmente muy eficiente: cálculo directo sin exponenciales.
- Evita la saturación del gradiente para $x > 0$.
- Estándar por defecto en **capas ocultas**.



Función	Fórmula	Función	Fórmula
ReLU	$\max(0, x)$	GELU	$\frac{x}{2} \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right)$
PReLU	$\max(0, x) + \alpha \min(0, x)$	ELU	$\begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \leq 0 \end{cases}$
Swish	$\frac{x}{1+e^{-x}}$	SELU	$\lambda \begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \leq 0 \end{cases}$
SoftPlus	$\frac{1}{\beta} \log(1 + \exp(\beta x))$	Mish	$x \cdot \tanh(\text{softplus}(x))$
Sigmoid	$\frac{1}{1+e^{-x}}$	SoftSign	$\frac{x}{1+ x }$

Subconjunto de Funciones de Activación

Función	Fórmula	Función	Fórmula
ReLU	$\max(0, x)$	GELU	$\frac{x}{2} \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right)$
PReLU	$\max(0, x) + \alpha \min(0, x)$	ELU	$\begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \leq 0 \end{cases}$
Swish	$\frac{x}{1+e^{-x}}$	SELU	$\lambda \begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \leq 0 \end{cases}$
SoftPlus	$\frac{1}{\beta} \log(1 + \exp(\beta x))$	Mish	$x \cdot \tanh(\text{softplus}(x))$
Sigmoid	$\frac{1}{1+e^{-x}}$	SoftSign	$\frac{x}{1+ x }$

Estimación de Hiperparámetros

Estimación de Hiperparámetros

- **Parámetros:** Son los pesos w y sesgos calculados por optimización.

Estimación de Hiperparámetros

- **Parámetros:** Son los pesos w y sesgos calculados por optimización.
- **Hiperparámetros:** Son los valores configurados externamente que afectan el aprendizaje de los parámetros.
 - Cantidad de capas.
 - Learning Rate (tamaño del paso en Gradient Descent).

Estimación de Hiperparámetros

- **Parámetros:** Son los pesos w y sesgos calculados por optimización.
- **Hiperparámetros:** Son los valores configurados externamente que afectan el aprendizaje de los parámetros.
 - Cantidad de capas.
 - Learning Rate (tamaño del paso en Gradient Descent).
- Ser cuidadosos con la elección de la **función de costo**.

Selección de Funciones y Optimizadores

Selección de Funciones y Optimizadores

- **En regresión:** Se suele usar **RMSE** o **MSE**.

Selección de Funciones y Optimizadores

- **En regresión:** Se suele usar **RMSE** o **MSE**.
- **En clasificación binaria:** Se suele usar **entropía cruzada binaria (BCE)**.

Selección de Funciones y Optimizadores

- **En regresión:** Se suele usar **RMSE** o **MSE**.
- **En clasificación binaria:** Se suele usar **entropía cruzada binaria (BCE)**.
- **En multiclase:** **Entropía cruzada categórica (CCE)**.

Selección de Funciones y Optimizadores

- **En regresión:** Se suele usar **RMSE** o **MSE**.
- **En clasificación binaria:** Se suele usar **entropía cruzada binaria (BCE)**.
- **En multiclase:** **Entropía cruzada categórica (CCE)**.
- **Solvers / Optimizadores:** Gradient Descent (GD), ADAM, RMSprop, etc.

Hiperparámetros de Arquitectura

Hiperparámetros de Arquitectura

- L : Cantidad de capas o profundidad de la red.

Hiperparámetros de Arquitectura

- L : Cantidad de capas o profundidad de la red.
- **Número de unidades por capa:** La entrada y salida dependen directamente de la definición del problema.

Hiperparámetros de Arquitectura

- L : Cantidad de capas o profundidad de la red.
- **Número de unidades por capa:** La entrada y salida dependen directamente de la definición del problema.
- **Estructura fija:** Rectangular (mismo número de unidades por capa oculta).

Hiperparámetros de Arquitectura

- L : Cantidad de capas o profundidad de la red.
- **Número de unidades por capa:** La entrada y salida dependen directamente de la definición del problema.
- **Estructura fija:** Rectangular (mismo número de unidades por capa oculta).
- **Estructura variable:** Piramidal (reducción progresiva de neuronas por capa).

Hiperparámetros de Entrenamiento

Hiperparámetros de Entrenamiento

- Learning Rate (LR)

Hiperparámetros de Entrenamiento

- Learning Rate (LR)
- Funciones de activación

Hiperparámetros de Entrenamiento

- Learning Rate (LR)
- Funciones de activación
- Batch size y Epochs

Hiperparámetros de Entrenamiento

- **Learning Rate (LR)**
- **Funciones de activación**
- **Batch size y Epochs**
- **Procesos de regularización** (Dropout, Weight Decay)

Hiperparámetros de Entrenamiento

- **Learning Rate (LR)**
- **Funciones de activación**
- **Batch size y Epochs**
- **Procesos de regularización** (Dropout, Weight Decay)
- Estrategias de prueba: *Trial and Error*

Hiperparámetros de Entrenamiento

- **Learning Rate (LR)**
- **Funciones de activación**
- **Batch size y Epochs**
- **Procesos de regularización** (Dropout, Weight Decay)
- Estrategias de prueba: *Trial and Error*
- **Partición de datos:**
 - 60% / 20% / 20% (pocos datos)
 - 90% / 5% / 5% (grandes volúmenes)

Hiperparámetros de Entrenamiento

- **Learning Rate (LR)**
- **Funciones de activación**
- **Batch size y Epochs**
- **Procesos de regularización** (Dropout, Weight Decay)
- Estrategias de prueba: *Trial and Error*
- **Partición de datos:**
 - 60% / 20% / 20% (pocos datos)
 - 90% / 5% / 5% (grandes volúmenes)
- **Técnicas de búsqueda:**
 - Grid Search
 - Random Search
 - Bayesian Search

Hiperparámetros de Entrenamiento

- **Learning Rate (LR)**
- **Funciones de activación**
- **Batch size y Epochs**
- **Procesos de regularización** (Dropout, Weight Decay)
- Estrategias de prueba: *Trial and Error*
- **Partición de datos:**
 - 60% / 20% / 20% (pocos datos)
 - 90% / 5% / 5% (grandes volúmenes)
- **Técnicas de búsqueda:**
 - Grid Search
 - Random Search
 - Bayesian Search

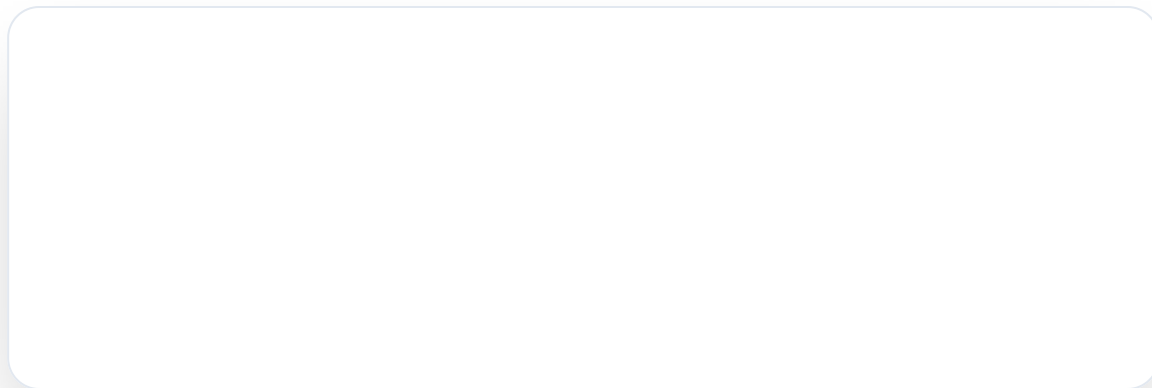
- **Grid Search:** Búsqueda exhaustiva en malla regular.

Hiperparámetros de Entrenamiento

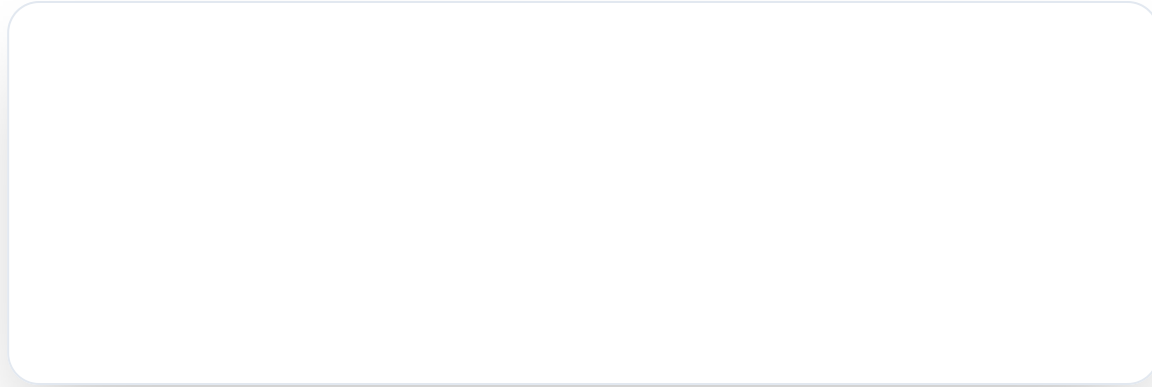
- **Learning Rate (LR)**
 - **Funciones de activación**
 - **Batch size y Epochs**
 - **Procesos de regularización** (Dropout, Weight Decay)
 - Estrategias de prueba: *Trial and Error*
 - **Partición de datos:**
 - 60% / 20% / 20% (pocos datos)
 - 90% / 5% / 5% (grandes volúmenes)
 - **Técnicas de búsqueda:**
 - Grid Search
 - Random Search
 - Bayesian Search
- **Grid Search:** Búsqueda exhaustiva en malla regular.
 - **Random Search:** Muestreo aleatorio en el espacio continuo.

Hiperparámetros de Entrenamiento

- **Learning Rate (LR)**
 - **Funciones de activación**
 - **Batch size y Epochs**
 - **Procesos de regularización** (Dropout, Weight Decay)
 - Estrategias de prueba: *Trial and Error*
 - **Partición de datos:**
 - 60% / 20% / 20% (pocos datos)
 - 90% / 5% / 5% (grandes volúmenes)
 - **Técnicas de búsqueda:**
 - Grid Search
 - Random Search
 - Bayesian Search
- **Grid Search:** Búsqueda exhaustiva en malla regular.
 - **Random Search:** Muestreo aleatorio en el espacio continuo.
 - **Bayesian Search:** Optimización probabilística guiada por evaluaciones previas.

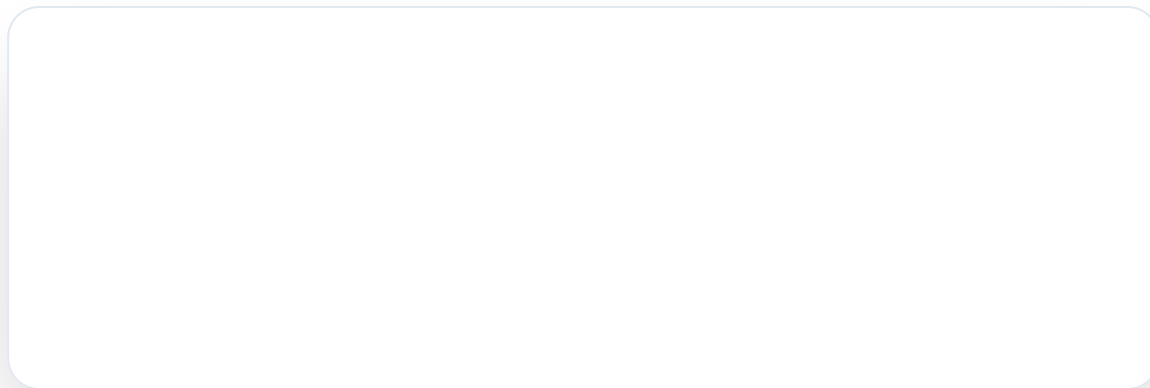


 Juguemos un rato con una NN



Juguemos un rato con una NN

Demostración interactiva en TensorFlow Playground





Juguemos un rato con una NN

Demostración interactiva en TensorFlow Playground



playground.tensorflow.org

Abrir en el navegador del salón y proyectar. Sin instalación.

Juguemos un rato con una NN

Demostración interactiva en TensorFlow Playground



playground.tensorflow.org

Abrir en el navegador del salón y proyectar. Sin instalación.

Reto 1: Con datos en espiral, ¿qué pasa si quitamos todas las activaciones no lineales?

Reto 2: ¿Cuántas neuronas necesita la capa oculta para separar dos círculos concéntricos?

Reto 3: Sube el *learning rate* a 1.0 – ¿qué le pasa a la frontera de decisión?

Datos Secuenciales

Datos Secuenciales

- ¿Qué son los datos secuenciales?

Datos Secuenciales

- ¿Qué son los datos secuenciales?
- **El dato actual depende de la información anterior.**

Datos Secuenciales

- ¿Qué son los datos secuenciales?
- **El dato actual depende de la información anterior.**
- **Dependencia temporal.**

Datos Secuenciales

- ¿Qué son los datos secuenciales?
- **El dato actual depende de la información anterior.**
- **Dependencia temporal.**
- *(Ejemplo: series financieras, índice bursátil Dow Jones a lo largo del tiempo)*

Ejemplos de Datos Secuenciales

Ejemplos de Datos Secuenciales

- Series de tiempo

Ejemplos de Datos Secuenciales

- Series de tiempo
- Señales de sensores

Ejemplos de Datos Secuenciales

- Series de tiempo
- Señales de sensores
- La voz (audio)

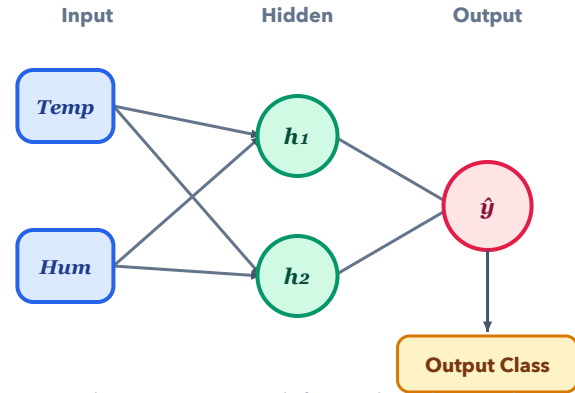
Ejemplos de Datos Secuenciales

- Series de tiempo
- Señales de sensores
- La voz (audio)
- Secuencia de genes (*gene sequences*)

Ejemplos de Datos Secuenciales

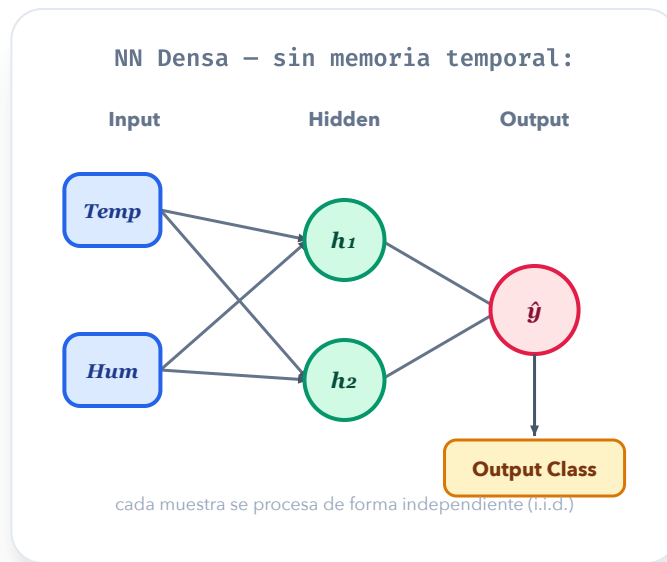
- Series de tiempo
- Señales de sensores
- La voz (audio)
- Secuencia de genes (*gene sequences*)
- Información climática

NN Densa – sin memoria temporal:



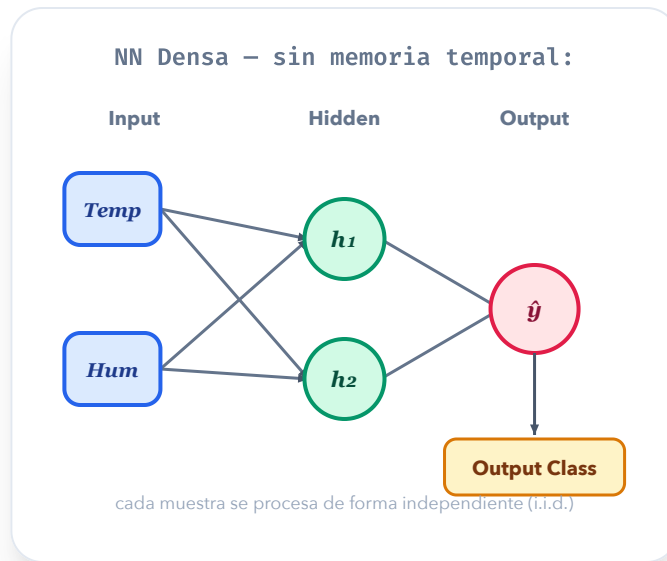
cada muestra se procesa de forma independiente (i.i.d.)

Limitaciones de las NN Tradicionales



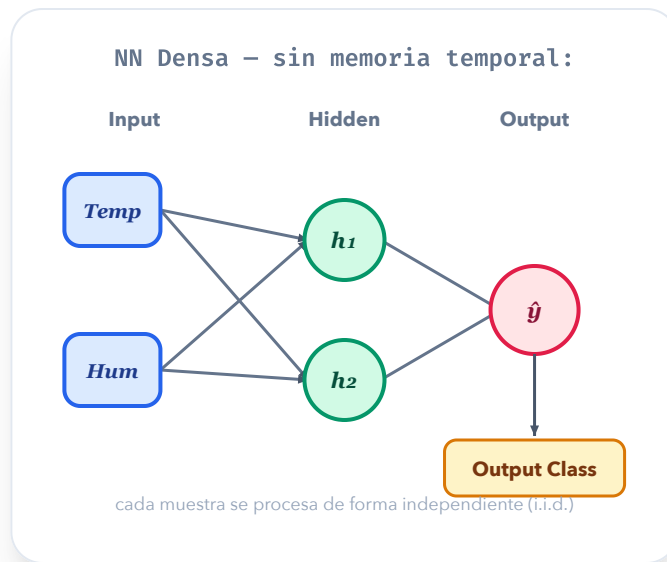
Limitaciones de las NN Tradicionales

- Una NN tradicional **asume que los datos no son secuenciales** y que cada punto de datos es independiente de otros puntos de datos (*i.i.d.*).



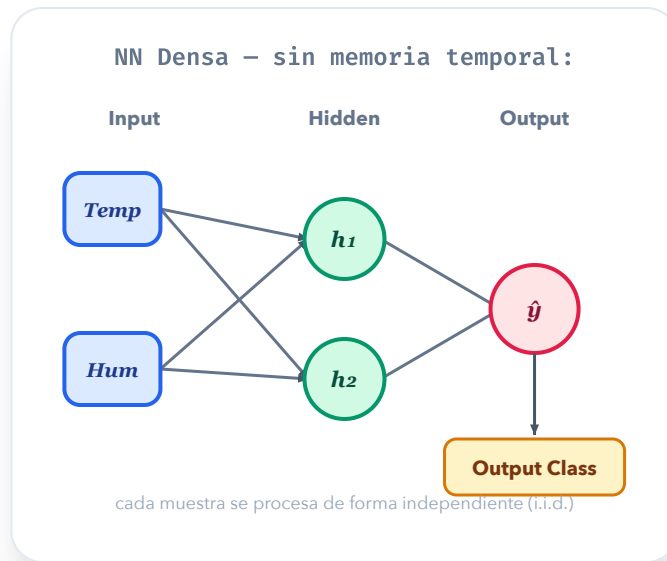
Limitaciones de las NN Tradicionales

- Una NN tradicional **asume que los datos no son secuenciales** y que cada punto de datos es independiente de otros puntos de datos (*i.i.d.*).
- **Ejemplo:** Temperatura - Humedad.



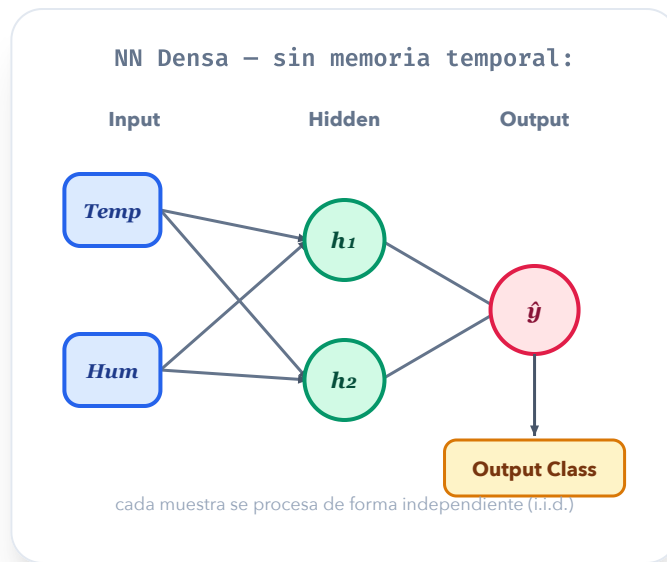
Limitaciones de las NN Tradicionales

- Una NN tradicional **asume que los datos no son secuenciales** y que cada punto de datos es independiente de otros puntos de datos (*i.i.d.*).
- **Ejemplo:** Temperatura - Humedad.
- **Carecen de memoria.**



Limitaciones de las NN Tradicionales

- Una NN tradicional **asume que los datos no son secuenciales** y que cada punto de datos es independiente de otros puntos de datos (*i.i.d.*).
- **Ejemplo:** Temperatura - Humedad.
- **Carecen de memoria.**
- **Sufren de gradientes que se desvanecen** (*vanishing gradients*) con secuencias largas.



Redes Neuronales Autorregresivas (NN-AR)

De serie temporal a problema
supervisado

Concepto de Ventana Deslizante ($n = 3$):

$X_1: [T_1, T_2, T_3]$

→ $Y_1: T_4$

$X_2: [T_2, T_3, T_4]$

→ $Y_2: T_5$

$X_3: [T_3, T_4, T_5]$

→ $Y_3: T_6$

La red densa aprende la dinámica temporal mapeando vectores fijos del pasado hacia el siguiente paso futuro.

Redes Neuronales Autorregresivas (NN-AR)

De serie temporal a problema
supervisado

- **Idea central:** Transformar una secuencia temporal $T_1, T_2, \dots, T_k$ en pares de entrenamiento (X, Y) mediante una ventana deslizante (lagged features).

Concepto de Ventana Deslizante (n = 3):

$X_1: [T_1, T_2, T_3] \rightarrow Y_1: T_4$

$X_2: [T_2, T_3, T_4] \rightarrow Y_2: T_5$

$X_3: [T_3, T_4, T_5] \rightarrow Y_3: T_6$

La red densa aprende la dinámica temporal mapeando vectores fijos del pasado hacia el siguiente paso futuro.

Redes Neuronales Autorregresivas (NN-AR)

De serie temporal a problema
supervisado

- **Idea central:** Transformar una secuencia temporal $T_1, T_2, \dots, T_k$ en pares de entrenamiento (X, Y) mediante una ventana deslizante (lagged features).
- **Formulación matemática:**

$$T(k) = f_{\theta}(T(k-1), T(k-2), \dots, T(k-n))$$

Concepto de Ventana Deslizante (n = 3):

$X_1: [T_1, T_2, T_3] \rightarrow Y_1: T_4$

$X_2: [T_2, T_3, T_4] \rightarrow Y_2: T_5$

$X_3: [T_3, T_4, T_5] \rightarrow Y_3: T_6$

La red densa aprende la dinámica temporal mapeando vectores fijos del pasado hacia el siguiente paso futuro.

Redes Neuronales Autorregresivas (NN-AR)

De serie temporal a problema
supervisado

- **Idea central:** Transformar una secuencia temporal $T_1, T_2, \dots, T_k$ en pares de entrenamiento (X, Y) mediante una ventana deslizando (*lagged features*).

- **Formulación matemática:**

$$T(k) = f_{\theta}(T(k-1), T(k-2), \dots, T(k-n))$$

- **Parámetro n (*window size*):** Cantidad de retardos pasados que alimentan la capa de entrada.

Concepto de Ventana Deslizante ($n = 3$):

$X_1: [T_1, T_2, T_3]$

→ $Y_1: T_4$

$X_2: [T_2, T_3, T_4]$

→ $Y_2: T_5$

$X_3: [T_3, T_4, T_5]$

→ $Y_3: T_6$

La red densa aprende la dinámica temporal mapeando vectores fijos del pasado hacia el siguiente paso futuro.

Redes Neuronales Autorregresivas (NN-AR)

De serie temporal a problema
supervisado

- **Idea central:** Transformar una secuencia temporal $T_1, T_2, \dots, T_k$ en pares de entrenamiento (X, Y) mediante una ventana deslizante (lagged features).

- **Formulación matemática:**

$$T(k) = f_{\theta}(T(k-1), T(k-2), \dots, T(k-n))$$

- **Parámetro n (window size):** Cantidad de retardos pasados que alimentan la capa de entrada.
- El modelo es una MLP densa entrenada con función de costo **MSE**.

Concepto de Ventana Deslizante ($n = 3$):

$X_1: [T_1, T_2, T_3]$

$\rightarrow Y_1: T_4$

$X_2: [T_2, T_3, T_4]$

$\rightarrow Y_2: T_5$

$X_3: [T_3, T_4, T_5]$

$\rightarrow Y_3: T_6$

La red densa aprende la dinámica temporal mapeando vectores fijos del pasado hacia el siguiente paso futuro.

Construyendo la Arquitectura en Keras

```
# 1. Regresión Lineal Clásica (1 sola neurona lineal)
model = Sequential([
    Input(shape=(10,)),
    Dense(1, activation='linear')
])
```

Construyendo la Arquitectura en Keras

```
# 2. Red Neuronal Profunda (MLP No Lineal para la serie temporal)
model = Sequential([
    Input(shape=(10,)),
    Dense(16, activation='relu'),      # 1ª Capa oculta: extrae patrones locales
    Dense(8, activation='relu'),       # 2ª Capa oculta: combina representaciones
    Dense(1, activation='linear')      # Capa de salida: pronóstico de temperatura
])
```

Construyendo la Arquitectura en Keras

```
# 3. Compilación y Entrenamiento con Gradiente Descendente
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae']
)

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=150,
    batch_size=32,
    verbose=1
)
```